COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 13, 2026	**DATE SUBMITTED:**	13/10/26
NAME:	ANDREW DOLOSO JR. RALPH FLORENZ DUE?…	**INSTRUCTOR:**	Engr, Jimlord Quejado

ACTIVITY NO. 12 [ALGORITHMIC STRATEGIES]

1. PROCEDURE OUTPUT

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

Strategy	Algorithm	Analysis
Recursion	Fibonacci Sequence	It calculates a term by repeatedly calling itself with smaller values until reaching base cases. This breaks down the sequence calculation into simple, self-referential steps.
Brute Force	Linear Search	It checks every element in the list one by one from start to end. This guarantees finding the target by exhaustively testing all positions without using shortcuts.
Backtracking	Depth-First Search (DFS)	It explores a path as far as possible and steps backward whenever it hits a dead end. This allows it to undo failed choices and try alternative routes.
Greedy	Selection Sort	It continuously picks the smallest remaining item and places it in its correct order. This makes the best immediate choice at each step without revising previous decisions.
Divide-and-Conquer	Merge Sort	It splits the array into two halves, recursively sorts them, and merges them back together. Dividing the problem makes sorting much simpler before combining the final result.

Table 12-1. Algorithmic Strategies and Examples

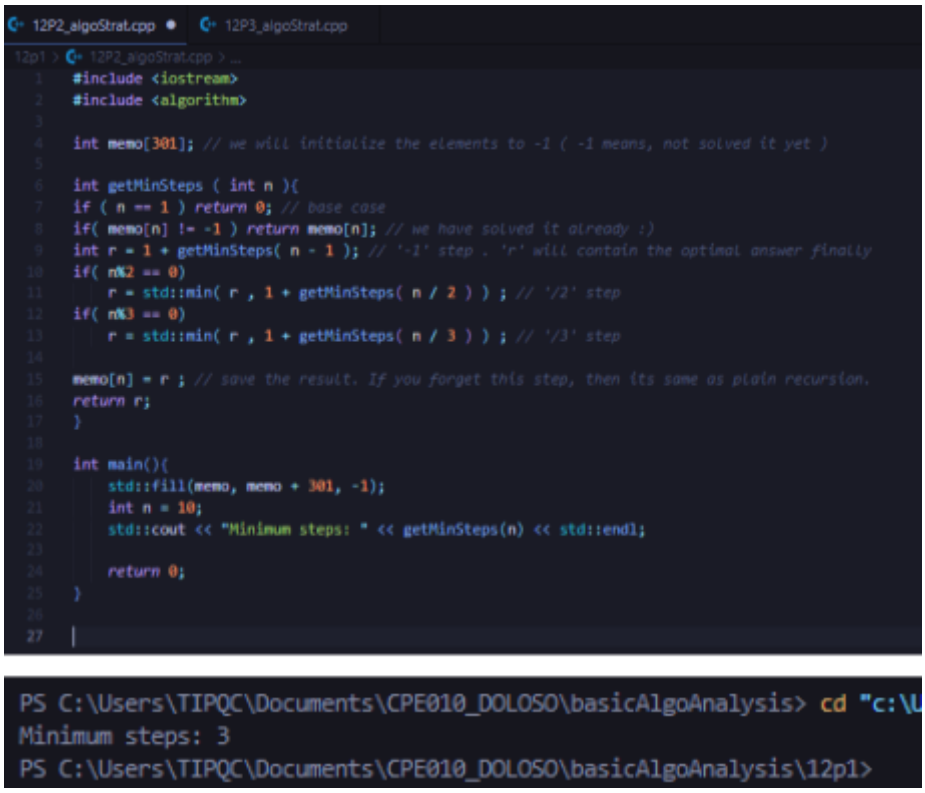
Screenshot	
Analysis	**DOLOSO** - Memoization makes recursion faster by saving Fibonacci numbers that have already been calculated in an array or table. When the same problem appears again, the program uses the saved result instead of calculating it again. This reduces the time complexity from O(2^n) to O(n), making the program faster and more efficient. **DUENAS** - In this implementation, the memoization technique avoids redundant calculations by keeping track of calculated Fibonacci values in memory. This top-down dynamic programming approach maintains the intuitive recursive structure while preventing duplicate work. As a result, the function call stack remains controlled and execution time is optimized for larger inputs.

Table 12-2. Memoization Implementation

Screenshot

12P2_algoStrat.cpp ● 12P3_algoStrat.cpp ×

12p1 > 12P3_algoStrat.cpp > ...

```cpp
#include <iostream>
#include <algorithm>
#include <vector>

int getMinSteps ( int n ){
    if (n <= 1) return 0;
    std::vector<int> dp(n + 1, 0);

    dp[1] = 0; // trivial case
    for (int i = 2; i <= n; i++)
    {
        dp[i] = 1 + dp[i - 1];
        if (i % 2 == 0) dp[i] = std::min(dp[i], 1 + dp[i / 2]);
        if (i % 3 == 0) dp[i] = std::min(dp[i], 1 + dp[i / 3]);
    }
    return dp[n];
}

int main(){
    int n = 10;
    std::cout << "Minimum steps to reduce " << n << " to 1: "
              << getMinSteps(n) << std::endl;
    return 0;
}
```

```
PS C:\Users\TIPQC\Documents\CPE010_DOLOSO\basicAlgoAnalysis> cd "c:\U
Minimum steps to reduce 10 to 1: 3
PS C:\Users\TIPQC\Documents\CPE010_DOLOSO\basicAlgoAnalysis\12p1>
```

Analysis

DOLOSO

- The bottom-up dynamic programming method does not use recursion. Instead, it solves smaller problems first and uses the results to solve bigger ones until it reaches the final answer. It runs in O(n) time, uses less memory for function calls, and works efficiently and reliably.

DUENAS

- The bottom-up dynamic programming method solves the problem by starting with f(0)and f(1), then calculating the next values until it reaches f(n). It saves the results in an array so they can be used easily in the next calculations. This method is faster and avoids the risk of stack overflow because it does not use recursion.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

Problem Title: Count the number of paths in a matrix with a given cost to reach the destination cell

Description:
Given an M × N integer matrix where each cell has a non-negative cost associated with it, count the number of paths to reach the last cell (M-1, N-1) of the matrix from its first cell (0, 0) such that the path has given cost. We can only move one unit right or one unit down from any cell, i.e., from cell (i, j), we can move to (i, j+1) or (i+1, j).

For example, consider the following 4 × 4 matrix where cost is 25. Two paths are having a cost of 25. A sample path has been given when the cost is 25.

4	7	1	6
6	7	3	9
3	8	1	2
7	1	7	3

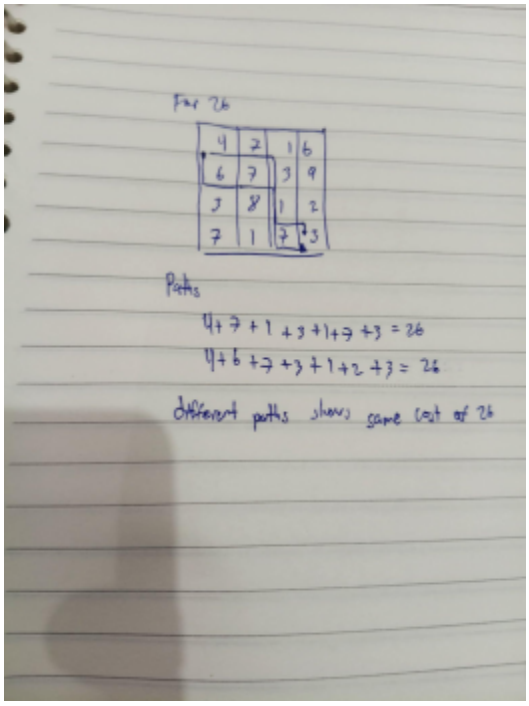
The idea is to use recursion. The problem has optimal substructure. That means the problem can be broken down into smaller, simple “subproblems”, which can further be divided into yet simpler, smaller subproblems until the solution becomes trivial.

Deliverables:

- **Pseudocode (+ Solving Problem by Hand)**

Pseudocode	Solving Problem by Hand

```
FUNCTION pathFind(row, column, remain)
  IF row or column is outside matrix return 0
  ELSE subtract current cell cost from remain
  IF remain < 0 return 0
  IF current cell is destination
    IF remain == 0 return 1
    ELSE return 0
  right = pathFind(row, column + 1, remaining)
  down = pathFind(row + 1, column, remaining)
  RETURN right + down
FUNCTION END
START
  Store matrix
  Print matrix
  User input of the required cost
  Print pathFind(0, 0, required cost)
END
```



- **Working C++ Code**
- **Analysis of working code**
- **Screenshots of demonstration**

```cpp
#include <iostream>
#include <vector>
#include <string>

int R, A;
std::vector<std::vector<int>> mat;
int calls = 0;

int countPaths(int i, int j, int cost) {
    calls++;

    if (i >= R || j >= A || cost < 0)
        return 0;

    if (i == R - 1 && j == A - 1) {
        if (cost == mat[i][j])
            return 1;
        return 0;
    }

    int down = countPaths(i + 1, j, cost - mat[i][j]);
    int right = countPaths(i, j + 1, cost - mat[i][j]);
    return down + right;
}

void printPaths(int i, int j, int cost, std::string path) {
    if (i >= R || j >= A || cost < 0)
        return;

    path += std::to_string(mat[i][j]);

    if (i == R - 1 && j == A - 1) {
        if (cost == mat[i][j])
            std::cout << "  " << path << "\n";
        return;
    }

    path += " -> ";
    printPaths(i + 1, j, cost - mat[i][j], path);
    printPaths(i, j + 1, cost - mat[i][j], path);
}
```

```cpp
int main() {
    int K;

    std::cout << "Enter number of rows and columns: ";
    std::cin >> R >> A;
    mat.assign(R, std::vector<int>(A));
    std::cout << "Enter the " << (R * A) << " matrix values:\n";
    for (int i = 0; i < R; i++) {
        std::cout << "Row " << (i + 1) << " (" << A << " numbers): ";
        for (int j = 0; j < A; j++) {
            std::cin >> mat[i][j];
        }
    }
    std::cout << "Enter the target cost: ";
    std::cin >> K;

    int r1 = countPaths(0, 0, K);

    std::cout << "\nRecursion      : " << r1 << " path(s), " << calls << " call

    std::cout << "\nPaths with cost " << K << ":\n";
    printPaths(0, 0, K, "");

    return 0;
}
```

```
Enter number of rows and columns: 5 5
Enter the 25 matrix values:
Row 1 (5 numbers): 4 7 1 6 2
Row 2 (5 numbers): 6 7 3 9 8
Row 3 (5 numbers): 3 8 1 2 7
Row 4 (5 numbers): 7 1 7 3 2
Row 5 (5 numbers): 2 4 5 8 9
Enter the target cost: 20

Recursion      : 0 path(s), 57 calls

Paths with cost 20:
```

Analysis	**DOLOSO** - The recursive function countPaths checks possible paths by moving down or right until it reaches the destination or exceeds the target cost. The printPaths function displays the paths that have the exact target cost. This method is easy to understand and shows how recursion works, but using memoization can make it faster for larger inputs by saving previous results.

	DUENAS - The simple uses recursion by moving down or right in each cell of the matrix. Although the code is easy to write and understand, it repeats the same calculations many times, making it slow as the grid gets bigger. This can lead to longer running times and more memory use because of the many recursive calls.

CONCLUSION:

DOLOSO

- In conclusion,on this HOA helped me understand different algorithm strategies, such as recursion, backtracking, brute force, greedy, and divide and conquer. In ILO A, I learned about these strategies by analyzing and finding algorithms that match each one. In ILO B, I learned how memoization (top-down) and tabulation (bottom-up) work in dynamic programming to find solutions more efficiently. Lastly, in the supplementary activity, I used recursive backtracking in a 4x4 matrix to find the correct path based on the given target value.

DUENAS

- In this HOA, I learned about different algorithm strategies in C++ and how to use them. I learned that brute force checks data step-by-step, divide and conquer breaks big problems into smaller parts (like Merge Sort), and recursion handles repeating tasks (like the Fibonacci sequence). I also discovered how dynamic programming uses top-down memoization and bottom-up methods to save past results, making programs run much faster. In the extra activity, I learned to balance speed, memory, and stack usage when picking the best algorithm. Overall, upgrading from basic recursion or greedy choices to dynamic programming helps us write much more efficient code.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.